

The study and data analysis of gravitational waves with matched filtering and convolutional neural networks

Noé Schärer¹

¹Université Paris Cité, Department of Physics - Internship at the University of Geneva, Département de Physique Nucléaire et Corpusculaire (DPNC), Gravitational Waves Group

August 3, 2024

Contents

1	Internship objectives, work methods and materials	
2	Mock Data Challenge	
2.1	Composition of the simulated data . . .	2
2.2	Objectives of the GW group within the MDC	2
3	Matched-filtering method for data analysis	
3.1	Objectives with the matched-filtering method	3
3.2	Basics of matched-filtering	4
3.3	Data production, pre-processing and method of computation of the SNR . . .	4
3.4	Results	5
4	Convolutional Neural Networks method for data analysis	
4.1	Objectives with the CNN and ML methods	5
4.2	Basics of a CNN for binary classification	5
4.3	Data production and pre-processing . .	6
4.4	CNN's training	7
4.5	Results for testing on unscaled MDC data and our unscaled and scaled data .	9
4.6	Results for testing on scaled MDC data	12
5	Comparisons between matched-filtering and CNNs	15
5.1	The need for other methods than matched-filtering for the future Einstein-Telescope	15
5.2	Comparisons between results with both methods	15
6	Global conclusion and outlooks	16
7	Annex	19

Abstract

This internship took place at the University of Geneva, in the Département de Physique Nucléaire et Corpusculaire (DPNC), within the gravitational waves group and under the direction of Assistant Professor Schramm.

This report has the objectives to transcribe the direct research work and results I produced with the group during my internship, which focused on the data analysis of the simulated data sets of gravitational waves produced for the Mock Data and Science Challenge associated to the future Einstein Telescope.

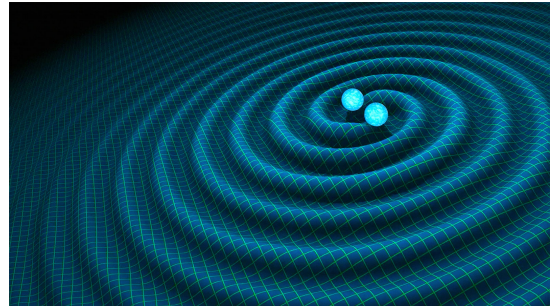


Figure 1: Artistic representation of a Compact Binary Neutron Stars System emitting gravitational waves

1 Internship objectives, work methods and materials

The global objectives were to study gravitational wave (GW) data with computer science and machine learning methods, such as matched filtering and convolutional neural networks (CNNs). For my code, see [14].

The first objective was to study in both a theoretical and practical way the general topic of neural networks, which I did mainly by following four different MOOC courses of the "Deep Learning" specialization on Coursera [1]. Then, I was fully integrated into the research group and was given the task to analyze the data produced for the Mock Data Challenge (MDC) for the future Einstein-Telescope (ET) with firstly the mathematically optimal method for signal detection known as matched-filtering. For this, I before had to follow a 2 days course on GW data analysis [5]. The third global objective was to analyze and augment simulated GW data produced by doctoral students from the group to prepare it for machine learning training, followed by building two CNNs in order to perform GW signal detection with another method than matched-filtering. While matched-filtering is mathematically optimal, other methods do have advantages especially with regards to the Einstein-Telescope data, as will be discussed later. I also participated in the creation of a poster, that we presented at the CHIPP 2024 Annual Meeting, which explained the objectives, methods and motivations of our work.

My working methods were to define objectives during weekly meetings of the group with Prof. Schramm and work semi-independently to develop new code and fix bugs. This included daily discussions with the doctoral student of the group, Ms. Sarah Baimukhametova, in order to solve both conceptual and practical issues and move on with the production of the code. I then presented my results at the next weekly group meeting.

My study materials were, in the first part of the internship and during my learning phase, mainly the MOOC courses on deep learning ([1]). I also used the GWOSC course [5] to learn about matched-filtering and libraries for coding and analyzing GWs data. For the rest of the internship, I focused on scientific literature in order to support my research and coding between CNNs

and GW data analysis. Many articles from forums were used to solve coding issues and bugs.

2 Mock Data Challenge

2.1 Composition of the simulated data

The MDC simulated data is composed of both direct GW signals and noise, and the typical scale of the strength of the signal is $10^{-22}/10^{-23}$ with noise usually having an expected amplitude of 2 to 3 orders of magnitude higher (see figure 2) at the future Einstein Telescope (ET). The noise origin is the relative displacement of the extremities of the arm's detectors, which are not caused by GW signals and the space-time deformations that accompany them, but rather can be due to cars or cows moving around the laboratory, earth's motion... Like a needle in a haystack, this makes it very hard to detect signals. For more information about gravitational waves and detectors, see [9] and [10].

The MDC data is composed of overlapping signals of mergers of binary black holes (BBH), binary neutron stars (BNS) and black-holes with neutron stars (BHNS). This means that their simulation was based on astrophysical and cosmological knowledge in order to produce populations of mergers reflecting the probabilities of each compact binary class, as well as intrinsic and extrinsic features. The future ET will have an extensive detection capability with respect to the range of frequencies of GW signals, causing it to have extremely high sensitivity. This will allow it to detect events at the rate of 1 per minute, compared to 1 per day for previous GW telescopes. The other side of the coin is that the data is thus fully constituted of overlapping signals, making it harder to extract individual merger events and infer astronomical parameters of the associated compact binary system (CBS). For more information, see [15].

2.2 Objectives of the GW group within the MDC

The objectives of the GW group in the DPNC at UniGE with respect to the MDC were first to detect GW signals using the matched-filtering (MF) method, followed by the improvement of the detection capabilities using

Data : signal and noise

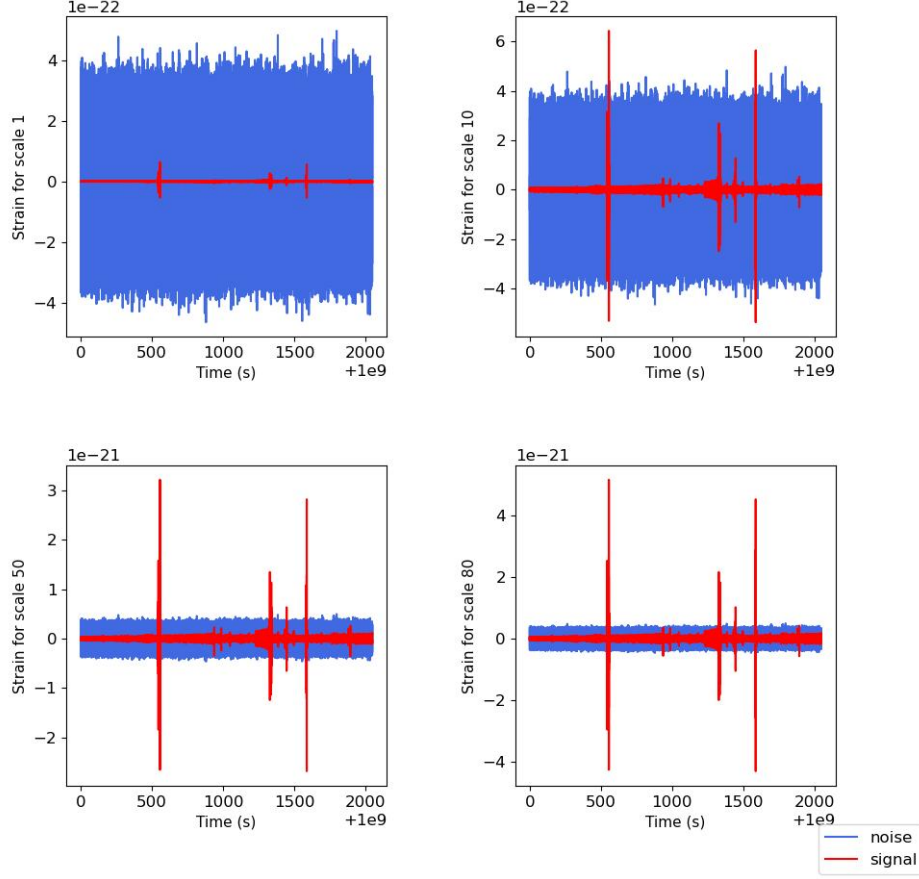


Figure 2: Signal and noise comparisons for the same level of noise, and with the signal at its expected strength (top left), or arbitrarily scaled up by a factor of 10 (top right), 50 (bottom left), or 80 (bottom right).

machine learning (ML) and CNNs. Therefore, we began by developing a Python MF algorithm to perform the recovery of high signal-to-noise ratio (SNR) signals within the time constraint given by the merger time of the different CBS in the simulation. But this method is known to be highly computationally expensive, and not well suited to overlapping signals. Thus, we then focused on the recovery of high SNR signals using solely CNNs and ML methods in order to adapt for the new ET detection capabilities. Ms. Baimukhametova approached this problem with 1D CNNs based on portions of time-series data and I with 2D CNNs based on spectrograms of portions of the same time-series, but with different time lengths. For these approaches, our objectives were for now to be able to differentiate between

two classes : (0 : pure noise) and (1 : merger).

3 Matched-filtering method for data analysis

3.1 Objectives with the matched-filtering method

I started my internship with the analysis of GW data produced for the ET MDC, using the method known as matched filtering (MF). I focused on the detection of high-SNR signals within a time constraint given by the merger times of the simulated events. Because I did not have access to large-scale computing resources

(like UniGe computing cluster), I performed every calculation on my computer and therefore was only able to evaluate the detection capability of my MF algorithm for the first parameters file of the MDC, out of a total of 1300 files, each containing the intrinsic parameters of every CBS whose signal is fully or partially contained in the 2048s associated data portion. After comparing our algorithms and results for this MDC data portion with Ms. Baimukhametova, and seeing almost identical results, she performed the rest of the calculations on the UniGe computing cluster for all of the MDC parameters files. Thus, the results with the MF method were available thanks to Ms. Baimukhametova.

3.2 Basics of matched-filtering

Matched filtering is the mathematically optimal solution for signal processing and detecting, with a linear filter, the presence of a template in unknown data composed of both signal and additive stochastic noise. It implies convolving the conjugate and time-reverse of a template to the unknown data and outputs a correlation value between the original template and the data. Doing this repetitive operation over all of the produced templates and across the time series allows you to scan the incoming amplitude gradually as it is produced. Matched filtering was proven to maximize the signal-to-noise ratio (SNR), and it is widely used in data analysis during signal processing. For example, the very first GW observation, on the 14th of September 2015 by the LIGO and VIRGO collaboration, was performed using template matching. It can also be used to approximate the parameter inference process, by comparing between templates similar to the signal once a high SNR event is recorded.

On the other hand, it is only the optimal solution for isolated/non-overlapping signals. It is basically really good at finding non-overlapping signals in noise or when focusing tightly around the merger, but has struggle with the inspiral phase for overlapping signals. Otherwise, you could apply MF with combined signals templates, but the computational cost grows exponentially with the number of templates (and MF with single signal templates is already very computationally expensive). Indeed, the nature of the data for the future ET GW detector will pose challenges for the uses of this method because of the highly overlapping characteristic of the signal, due to its sensitivity and detection rate. Another important limitation associated with MF is that

it is computationally expensive because it involves convolving huge data banks of templates with the unknown data. Therefore, new methods like CNNs are now being developed to speed up and improve signal processing performance, notably with regard to the extreme augmentation of the rate of events with the future ET.

3.3 Data production, pre-processing and method of computation of the SNR

For the production of the templates, we used the "get_td_waveform" function from the pycbc library [12] for the waveforms (WFs) generation with the approximant called "IMR-PhenomD" [4], [3]. This approximant uses hybrid WFs which compute the value in different regimes with varying methods. Part of the inspiral is generated with Effective One Body (EOB) models, which are an extension of the Post-Newtonian (PN) models with higher order terms calibrated to Numerical Relativity (NR) data simulations. Then, the last part of the inspiral, the merger and the ringdown use strictly NR to solve the non-linearised Einstein's field equations. They finally connect the WFs from both regimes to create the hybrid functions. To sample the parameter space and create a data bank of templates, Ms. Baimukhametova simply took the parameters given by the parameters files in the MDC, and produced the associated WFs with a low frequency cutoff of 7 Hz and the chosen pycbc function.

We then projected the WFs onto each of the three ET arms (E1, E2, E3) using the "project_wave" function from the "detector" module of the pycbc library ([11]). These few steps allowed us to create a bank of thousands of WFs, ready to perform matched-filtering of the MDC data on each of the three arms of ET. The global SNR result s_{global} , for s_i the SNR computed on arm i , is given by :

$$s_{global} = \sqrt{\sum_{i=1,2,3} s_i^2} \quad (1)$$

For our single signal MF study, we used 90s windows of the data with 45s overlap, which are then each converted into a power spectral density (PSD) time series. Then, you convolve every incoming PSD with all of the template bank to calculate SNR values corresponding to MF calculation between the data and each template. We chose to then keep only the highest SNR value, associated to the match with the highest probability. If the difference in time between the selected SNR detection

and the real peak time of the WF in the MDC data was below 0.1s, and the SNR value above 10, we considered it a true detection.

3.4 Results

One of the first research objectives of the team, regarding the MDC, was to evaluate the fraction, per parameters file of the MDC, of mergers events with SNRs above a certain value. Most of the values found during MF being around 5, we chose to set the threshold associated to a confirmed detection at 10.

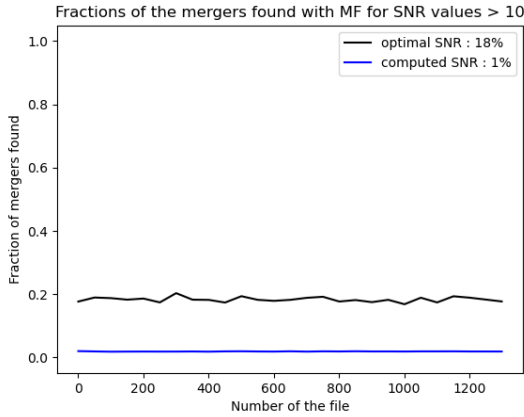


Figure 3: Fractions of mergers found per MDC file with a SNR threshold value of 10

Even with such a low SNR threshold value, we can see that the fraction of the mergers found per file is still very low and has a mean of around 1% (see figure 3). Also, about 72% of the detections are BBH, 27% BHNS, and less than 1% BNS, even though we use large 90s windows of the data. We know that the structure of the WFs of BNS and BHNS (see [9] and [10]) imposes large time scale window in order to perform MF, which means that it is large enough for BBH mergers detections, but still too short for other CBS. The detection performance is worsened when compared to the optimal SNR value, which is only of 18% for the same threshold at 10. This figure confirms that single signal MF is clearly a non usable method for GW mergers detections with overlapping signals. Added to its associated requirements of high computational resources, MF performed with single signals templates is simply not an option to perform signal detection for the future Einstein-Telescope. Performance could be recovered

with multiple combined signals, but that would make the computing cost grow exponentially with the number of templates combined. Thus, new methods like 1d or 2d CNNs need to be implemented for GWs detections with the arrival of the new era of telescopes.

4 Convolutional Neural Networks method for data analysis

4.1 Objectives with the CNN and ML methods

During the second and main part of my internship, I focused on the study of detection and differentiation capabilities between noise and mergers using 2D CNNs and ML methods, preceded by data pre-processing. The objective was to evaluate the possibility of transferring feature extraction capabilities between CNNs trained on scaled data and unscaled data, both overlapping (MDC) and non-overlapping (our data), and tested on all scales and types of data sets. Thus, the goal was not, for now, to be able to have a great performance for CNNs trained and tested on unscaled data, notably because of the lack of a sufficient statistics in the non-overlapping data. In order to reach these objectives, I built and trained CNNs to differentiate between the two classes, with high accuracy, within the different kinds and scales of data sets.

4.2 Basics of a CNN for binary classification

A convolutional neural network is a neural network characterized by the feed-forward flow type, which means information goes uni-directionally between layers only from the input nodes, through the hidden nodes and to the output nodes. They are especially designed for image classification or object detection. These two tasks can, for example be respectively, classifying dogs and cats or detecting a cat's position in an image. In this internship, I only performed image classification.

The training method of CNNs is based on backpropagation which uses standard gradient descent on a loss function. It works by attributing to each node a non-linear matrix function $\hat{y}_{i,j}$ composed of weight matrices $(w_{i,j}, b_{i,j})$ (see eq. 2), computing the results of the functions of each node with respect to the input data or the previous node output, and then doing gradient

descent to update these weight matrices and minimize loss. Here, the layer is designated by i and the node index within this layer by j , with σ the activation function :

$$\hat{y}_{i,j} = \sigma(w_{ij}y_{i,j-1} + b_{i,j}) \quad (2)$$

For a given problem, you define a loss function and try to minimize this loss function by gradient descent. For example, the loss function I used to differentiate between the two classes defined as (0 : noise) and (1 : merger) is called binary-cross-entropy L and is widely used for binary classification :

$$L = -\frac{1}{2} \sum_{i=0,1} y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i)) \quad (3)$$

This simplifies for labels $i = 0, 1$ to :

$$L = -\log p_1 \quad (4)$$

Therefore, the goal of gradient descent in the context of binary classification is to concentrate the probability density function p_1 around a certain point of the highly dimensional parameter-space of the activation's functions, in order to minimize loss L and thus perform back-propagation. At every step of back-propagation, the CNN updates its weights matrices for each node at each layer. By this process, it learns progressively to predict more accurately the correct output and thus the right class of objects.

4.3 Data production and pre-processing

The data on which the CNN was trained was produced by Ms. Baimukhametova with the gwpy library ([6], [7]) on the UniGe computing cluster. It is composed of non-overlapping GW signals on which was added additive stochastic noise. This means that when a signal ends, a new one is concatenated to the previous one. We also produced different scales of GW signals, which means that the strain solely associated to the GW propagation was scaled by values of (1, 10, 20, 50, 80), leaving the noise untouched (see figure 2). This basically produces signals that have increasing SNRs with respect to the augmentation of the scale factor.

M1 ($M_\odot$)	[4, 50]
M2 ($M_\odot$)	[4, 50]
Distance (Mpc)	[200, 60000]
DEC ($^\circ$)	[-90, 90]
RA ($^\circ$)	[0, 360]
Polarization angle	[0.05, 7]

Table 1: Waveform generation priors

The population of the CBS of this data is solely composed of BBH, which were the type of binary systems that had the highest SNRs during the MF analysis and constituted the largest fraction of CBS detected (72% of the mergers detected were BBH). This allows use to ensure high detection capabilities, regarding of the CNNs development level that we are at. The WF generation priors (see table 1) were scanned across with uniformly random probability densities. This implies there is, for now, no astronomical and cosmological physics implied in the data production in order to create a parameter-space of the CBS that reflects their true natural properties. In the end, there was 1176 files, each lasting 2048s (+34 min.), for a total of 6348 mergers.

The production of the spectrograms of our data (see figure 4 for a random single merger : RSM) is based on the fact that we wanted first to be able to differentiate between pure noise and merger, before adding in the future more classes like pre-merger, ringdown and type of CBS. Therefore, for each merger, I took a 16 second window centered around the merger time (peak time of the signal) and made a spectrogram of this time-series section using the over-dense and short-duration spectrogram method from the gwpy library [8]. This spectrogram method produces richer feature output than non-overlapping methods thanks to its Fast-Fourier-Transform (FFT) calculations which are highly-overlapping in order to over-sample the frequencies. This allows to extract more precise frequency features of the time series than traditional FFT methods and gives an easier input for learning to the CNN. The parameters I chose are $fftlength = 0.25$ s and $overlap = fftlength/2$ s. To have a balanced dataset, I also took the 16s of the noise files right before the merger window and made a spectrogram of it. For every file of data, one spectrogram file was produced, each containing around 6 mergers and 6 noise spectrograms, amounting up to the total of 1176 files containing 6348 mergers and $2 \times 6348 = 12696$ spectrograms. All the files were saved as 3d arrays under .npy format. The production of MDC spectrograms was done in the same way, using the same conditions and parameters (see figure 5). But the first difference is that because of the presence of overlapping signals, much more spectrograms were produced, with a total of 69781 mergers and thus $2 \times 69781 = 139562$ spectrograms. The second difference, which explains the difference in intensities between spectrograms of MDC data (see figure 5)

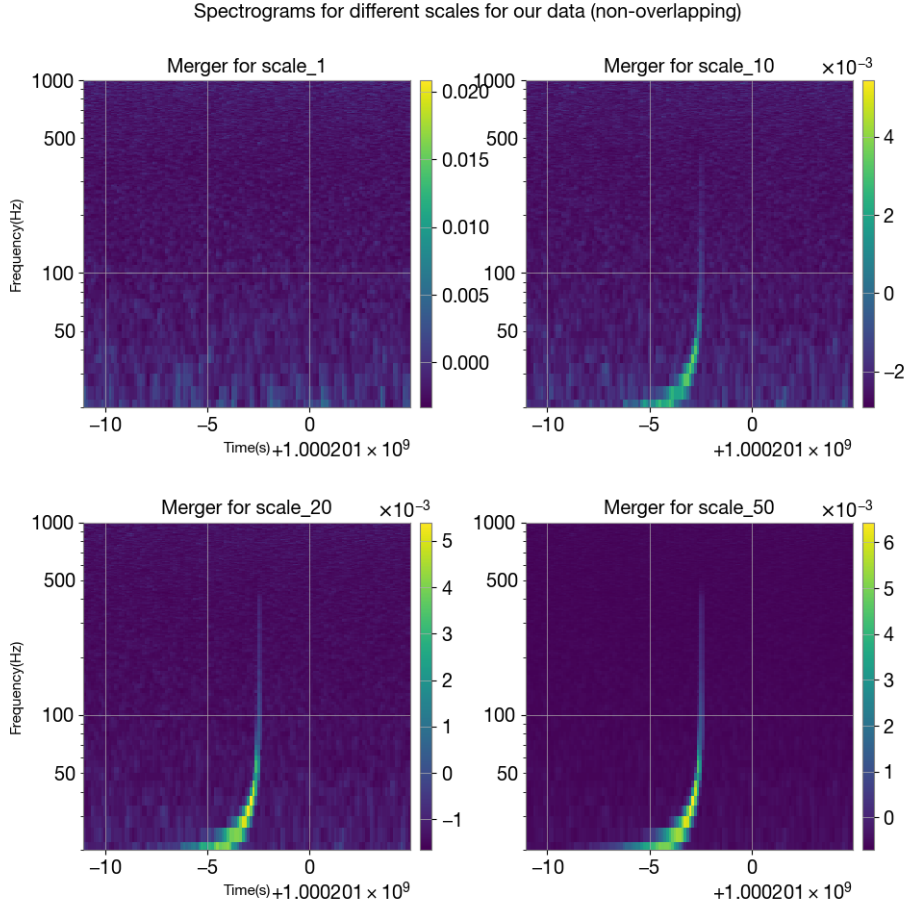


Figure 4: Spectrogram of a random single merger (BBH) for different scales of the signal for our data

and our data (see figure 4), is that the composition of the CBS for MDC data is distributed among BBH, BNS and BHNS, where ours is solely composed of BBH. BBH mergers being the ones with the highest SNRs. This, as well as the varying WF structures for different CBS types, explains why a plot of a random single merger for our data appears more clearly than for MDC data. But for MDC data, doing a plot of a random single merger is more likely to fall on BNS or BHNS, explaining the low intensity associated to the norm of the FFT. To showcase these differences, I therefore put a spectrogram merger randomly between these three classes. Finally, because BBH mergers produce lower frequency GWs, I decided to cut every spectrogram to a frequency of a 1000 Hz, making the CNN's learning, feature extraction and detection processes easier. In return, it will be harder to

detect BNS and BHNS mergers when tested on MDC data set. Indeed, their optimal SNR value is not much lower, but the difference lays in the structure of their WFs, which are much longer in-band and thus to get the full SNR, you need to integrate over a much longer time window. Therefore, 16s works well for BBH as I will show later, but one would need much longer window to see BNS and BHNS signals with reasonable sensitivity. As our main objective is, for now, to reach a high differentiation capability between noise and high SNRs mergers, this disadvantage weights low in the balance.

4.4 CNN's training

The CNN I constructed is a Res-Net 50 (for more information : [13]) composed of both convolutional

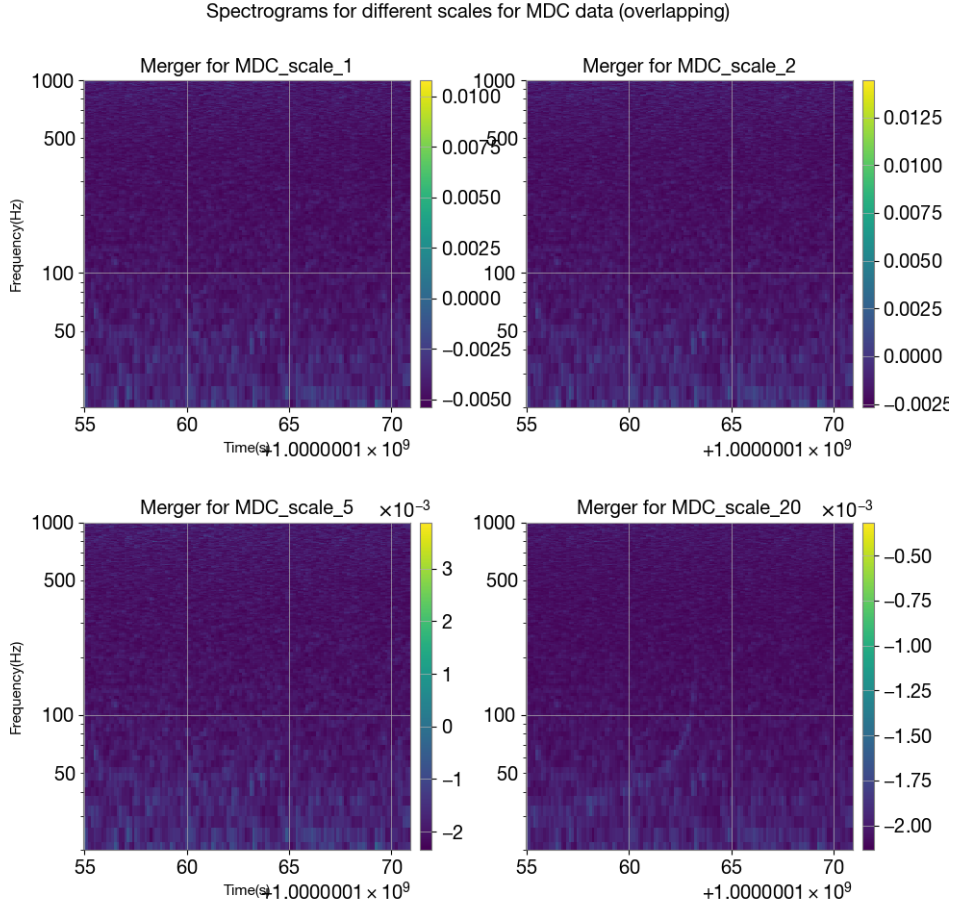


Figure 5: Spectrogram of a random single merger (BBH, BNS or BHNS) for different scales of the signal for MDC data

and identity blocks (see Annex for the full architecture). Deep neural networks (DNNs) tend, as they become deeper, to have a saturation in their accuracy which then may decrease due to the complexity of the optimization process during training. But DNNs are necessary to map complex relations between diverse and intricate inputs and their associated output labels and thus perform accurate predictions. This is also true for image classification as objects of the same class can come in various shapes, colors and other characteristics. Therefore, Residual-Networks (Res-Nets) introduce residual blocks (or identity blocks), which allow to "skip" a few layers by adding the results of both one or more convolutional layers with an identity layer. This still permits the recording and

learning of object features and it has, in the end, better information flow between layers than traditional DNNs.

My Res-Net 50 (see Annex for the full architecture) begins with a traditional shallow CNN composed of layers that are conv2D, batch-normalization, relu activation and max-pooling. Then, it is followed by alternations of convolutional and identity blocks finally ended by an average-pooling layer and a dense layer. The structure of the model was highly inspired by the Res-Net 50 model I developed for an exam of the ML and Deep Learning MOOC courses [1], with parameters (like kernel and filter sizes) adapted to the spectrograms of the GW data-sets.

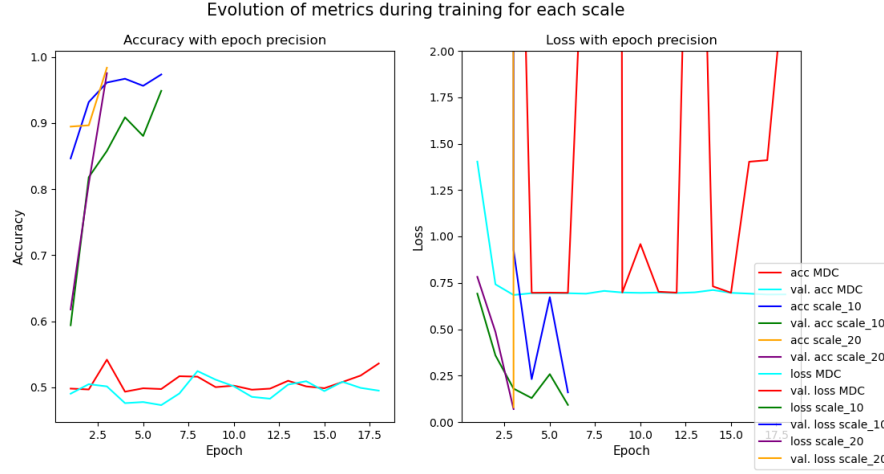


Figure 6: Evolution of the accuracy and the loss at each epoch, both for the training and evaluation sets, during training for MDC unscaled and a few of our data scales (10, 20)

The CNN training was performed with different values for its parameters, like kernel and filter sizes, or dropout and L2 regularization factors, as well of the learning-rate. For the task of binary classification with our spectrograms data-sets, dropout and L2 regularization were finally set to 0. I chose to do so because of only having access to limited calculation resources and the slow convergence of versions with these factors being different from 0. This allowed me to quickly build up CNNs trained on data with different scale-factors of the GW signals while still obtaining high accuracy. The training fraction was set to 0.7, giving 8887 spectrograms to train on and 3809 to test on for each scale factor of our data.

4.5 Results for testing on unscaled MDC data and our unscaled and scaled data

In order to evaluate the possibility of feature extraction transferring capability between CNNs trained on scaled data and unscaled data, I first needed to train a CNN on different data sets. Therefore, different versions of the CNNs were created, all with the same structure (Res-Net 50) and parameters, all trained on a specific data set. For overlapping data sets (MDC), there was only MDC unscaled. For non-overlapping data sets (our data), there were four scales (10, 20, 50, 80). The evolution of the important metrics (accuracy and loss, both for training and validations) are represented on the figure 6 for a few data sets. You can see that for

non-overlapping and scaled data, accuracy and loss, both for training and validation, quickly converge. But that is not the case for unscaled data, both non-overlapping (our data, non represented of figure 6) and MDC overlapping data. One of the reasons for this is the lack of a sufficient quantity of data, which will be discussed in section 6.

The first logical result to look for was to evaluate the performance of each CNN mentioned above, tested on itself (see figure 7). This performance is represented by Receiver Operating Characteristic (ROC) curves which evaluate the true positive rate vs false positive rate, and it's Area Under Curve (AUC) : $AUC = 0.5$ is as bad as a random classifier and $AUC = 1.0$ is a perfect classifier, always with regard to the tested data.

It is shown that training and testing on unscaled MDC data, even though the presence of a high number of mergers (69781) and $2 \times 69781 = 139562$ spectrograms, does not allow the CNN to pick up low intensity FFT features and thus has an AUC near 0.5. This is due to the fact that most merger events in unscaled data, both for MDC and our data, has a low optimal-SNR value, and therefore is hard to detect even with single signal MF. The CNN with our unscaled data had quite the same performance, because of the low quantity of inputs, so I decided not to include it. But on the contrary, for scaled and non-overlapping data, the CNN is extracting the spectrograms features really well, which is rep-

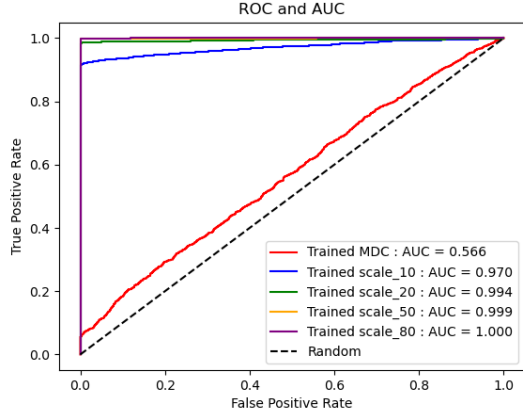


Figure 7: ROC curves for different scales of non-overlapping data and MDC, tested on themselves

resented by high AUC values for scales (10, 20, 50, 80).

The second step in evaluating the possibility of transferring feature extraction capabilities between different signal scales was to calculate the ROC curves and AUC for the four CNNs, trained on non-overlapping signals of our data sets, and tested on other scales (see figure 8) and unscaled MDC data (always used for reference). For all of the training scales, we can clearly see that unscaled MDC data is not picked up at all by the CNNs and has results comparable to ones with random classifiers. Therefore, we first of all conclude that data sets constituted of unscaled and overlapping signals are out of range of detection for CNNs trained on scaled and non-overlapping signals. This tendency is well shown on the frequencies of the CNN output plot (see figure 10). Indeed, a great comparison of its curve to the plot of a Fisher Matrix can be done for training scales (10, 20). A Fisher Matrix is a way of representing the information in stochastic models, which is the case of gradient descent and CNNs. This is especially true for classification when they have not learned to significantly differentiate between the classes and thus have low performance. Then for training scales (50, 80) and still MDC unscaled data, the CNNs are just random guessing all of the spectrograms to the same output to minimize loss and maximize accuracy, showing a frequency plot tightly centered around 0 or 1 and making it a perfect random classification process. Indeed, for training scale 50 everything is seen as noise, and for training scale 80, everything is

seen as signal. This last fact can be due to the weights initialization of the CNN causing a random guess to 1, which for a perfectly balanced data set implies an AUC of 0.5. In conclusion, there is simply no good feature extraction transferring capability between training on scaled non-overlapping data and testing on unscaled overlapping data.

The next interesting thing, for training scales (10, 20, 50) (see figures 8 and 9), is that the AUC steadily increases with regard to the testing scale for non-overlapping data, reaching values of nearly 1 for scales up from 10. Non-overlapping scale 1 still has better performance than a random classifier, which means there is some transferring capability of feature extractions between CNNs trained on high SNR signals (scales bigger than 1) and tested on "normal" SNR signals. We could now make the supposition that with more scaled data for training (for now only around 12000 spectrograms per scale), the AUC value for testing on scale 1 could increase and reach a better value. Also, we can see that for all of the CNNs except the one trained on scale 80 and tested on scales above 1, there is absolutely no issue with transferring feature extraction capabilities between scales, which means they do a great job differentiating between the two classes noise and merger. This is represented by slowly increasing and near 1 value ROC curves, as well as near 1 value for their AUC. These results are clearly confirmed by the frequency of the CNN output (see figure 10). We can see a great differentiation, on different training scales (10, 20, 50) and for all of the tested scales above 1, between the two classes. Indeed, there is in most cases the appearance of two boxes that get more and more tightly centered around two values as the training scale grows (very visible for tested scale 10). We can also recognize the fact that tested scale 80 always looks like mergers for trained scale 10, and then gets more balanced between the two classes. Finally for scale 1, we see that everything is almost always considered as noise. Therefore, we can conclude that the transferring capabilities of features extraction are high within the range [10, 50], which means between high SNRs mergers of the same CBS type.

Finally, for the CNN trained on scale 80, we can see clear issues with transferring its capabilities of feature extraction during testing to other scales except scale 80 (where it does a perfect job). Indeed, there is low AUC values (see figure 9) and inadequate ROC curve

ROC and AUC trained on different scales for SCALES

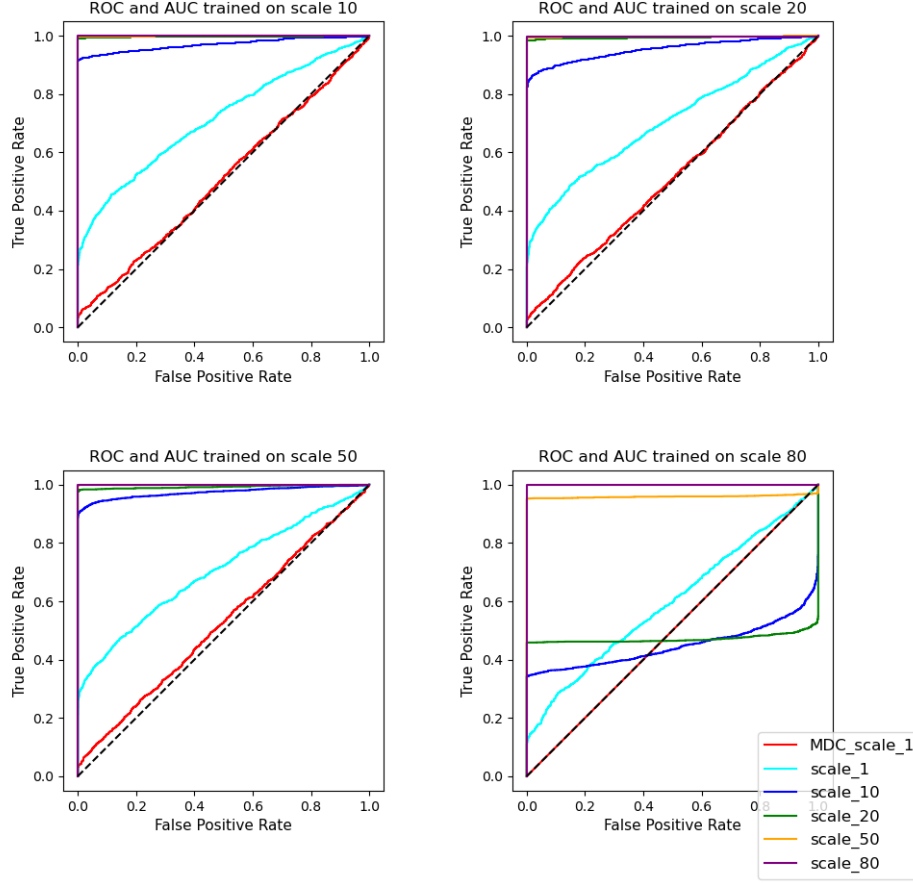


Figure 8: ROC curves for training on different scales of non-overlapping data, tested on different scales of non-overlapping data and MDC unscaled

Tested scales	AUC for SCALES			
	Trained scale 10	Trained scale 20	Trained scale 50	Trained scale 80
MDC scale 1	0.51	0.51	0.52	0.5
scale 1	0.71	0.7	0.7	0.6
scale 10	0.97	0.95	0.97	0.45
scale 20	1.0	0.99	0.99	0.47
scale 50	1.0	1.0	1.0	0.96
scale 80	1.0	1.0	1.0	1.0

Figure 9: AUC values for training on different scales of non-overlapping data, tested on different scales of non-overlapping data and MDC unscaled

(see figure 8). This is due to the fact that the merger spectrograms that it learned on have an intensity that is way too high for it to classify spectrograms from other scales as mergers, and it just sees noise instead. We can see the representation of this phenomena on

the frequency of the CNN output plot (see figure 10), where scales below 50 are interpreted as always noise or classified with the high tendency to choose the class that has the highest number of elements to minimize loss and maximize accuracy. This first point explains why for non-overlapping data and scales (1, 10, 20), the output is almost always (0 : noise). The second point explains why MDC unscaled data is always considered as merger, possibly because of the overlapping nature of the signals, or because of random choices solely to an output of 1 due to weights initialization. The scale 50, which has signals SNRs close to scale 80, performs well on trained scale 80.

In conclusion, the CNN architecture Res-Net 50 al-

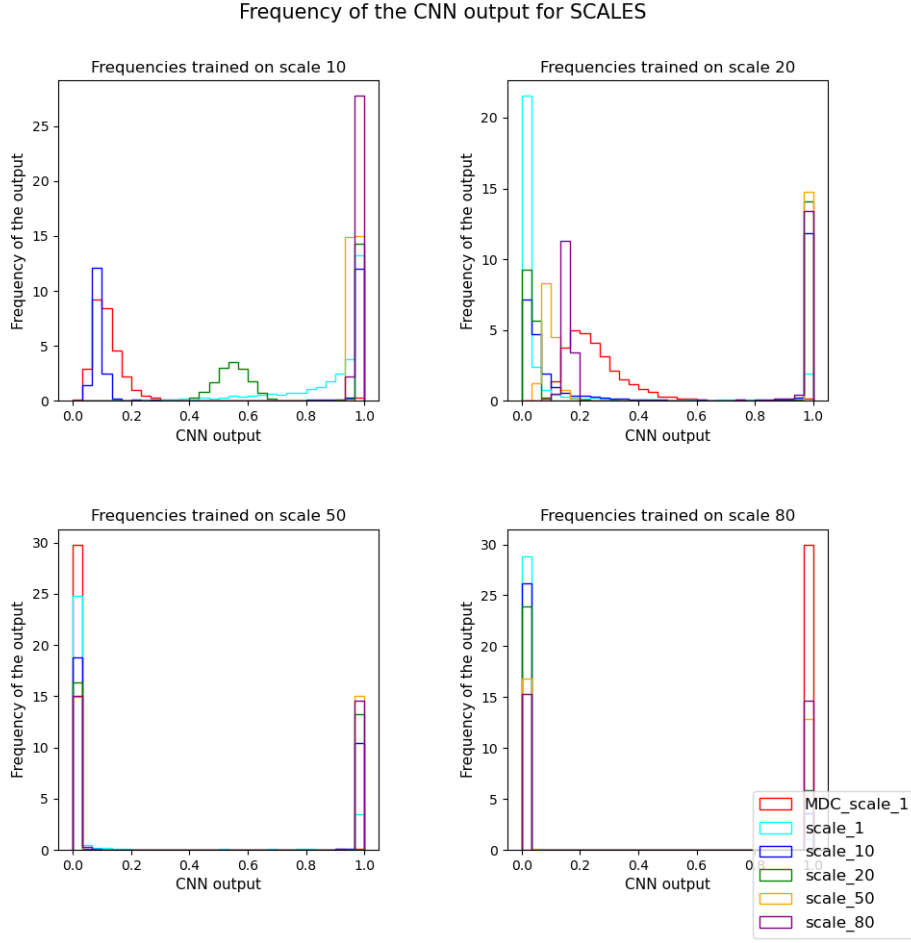


Figure 10: Frequencies of the CNN’s output for training on different scales of non-overlapping data, tested on other scales of non-overlapping data and MDC unscaled

allows us to have a great performance for transferring feature extraction capabilities between different scales ranging from 10 to 50 for non-overlapping signals. This supports the statement that mid SNRs non-overlapping signals are still being picked up by CNNs trained on high SNRs non-overlapping signals.

4.6 Results for testing on scaled MDC data

After evaluating the transferring capability of feature extraction for non-overlapping signals (our data scaled and unscaled) and unscaled MDC data, we can get interested in asking the following question : can CNNs trained on different scales of non-overlapping signal

data still detect overlapping signals of varying SNR values ? The short answer is yes, but only for high SNR merger events. Another question could be : can CNNs perform better than single signal MF on the same input data ? Yet again, the short answer is yes, but only slightly for the small quantity of data they were trained on.

The first results to look for in order to answer this question are the ROC curves (see figure 11) and AUC values (see figure 12). We can see on figure 12 that for training scales (10, 20, 50), all of the tested scales larger than 1 of MDC data have AUC values very close to 1. This is well represented by the associated ROC curves (see figure 11) which have clearly a near

ROC and AUC trained on different scales for MDC

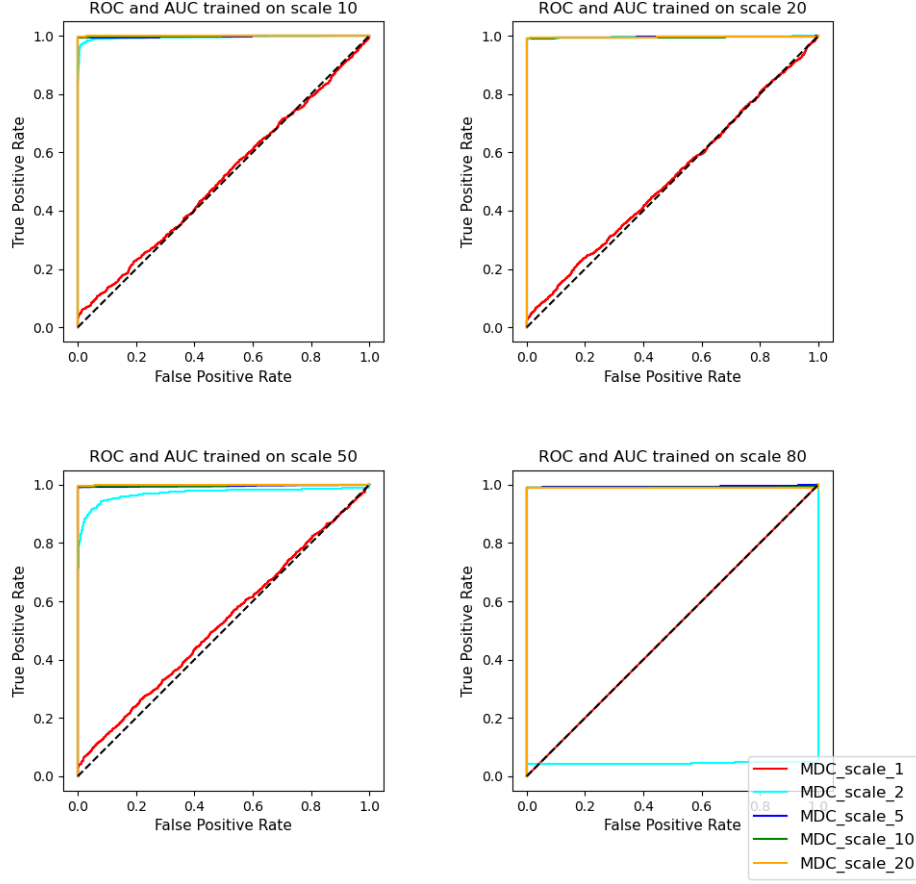


Figure 11: ROC curves for training on different scales of non-overlapping data, tested on scaled and unscaled MDC data

Tested scales	AUC for MDC			
	Trained scale 10	Trained scale 20	Trained scale 50	Trained scale 80
MDC scale 1	0.51	0.51	0.52	0.5
MDC scale 2	0.99	1.0	0.97	0.85
MDC scale 5	1.0	1.0	1.0	0.99
MDC scale 10	1.0	0.99	1.0	0.99
MDC scale 20	1.0	1.0	1.0	0.99

Figure 12: AUC values for different scales of MDC overlapping data

constant value of 1 for true positive rate (TPR) for every false positive rate (FPR) value. We can see, for tested MDC scale 2, that there is excellent identification of mergers or non-mergers for trained scales (10, 20) with a small decline in performance when evaluated on the CNN trained on scale 50. This is the same tendency

as our non-overlapping data : the CNN starts to label mergers as noise as the SNRs of the signals it was trained on are so high compared to the SNRs of the tested data. This is obviously seen on the frequency of the CNN's output (see figure 13), on which for trained scale 10 the CNN is absolutely certain when there is a merger (high peak around 1), but then is uncertain about classifying non-mergers because of the presence of overlapping signals. For higher training scales, it basically flips to being sure when it is non merger, because of the high values of the SNR's mergers it was trained on and thus sees overlapping merger signals as noise, and struggles with the identification of mergers : there is no concentration of the CNN's output around 1, the frequencies are distributed with a large deviation.

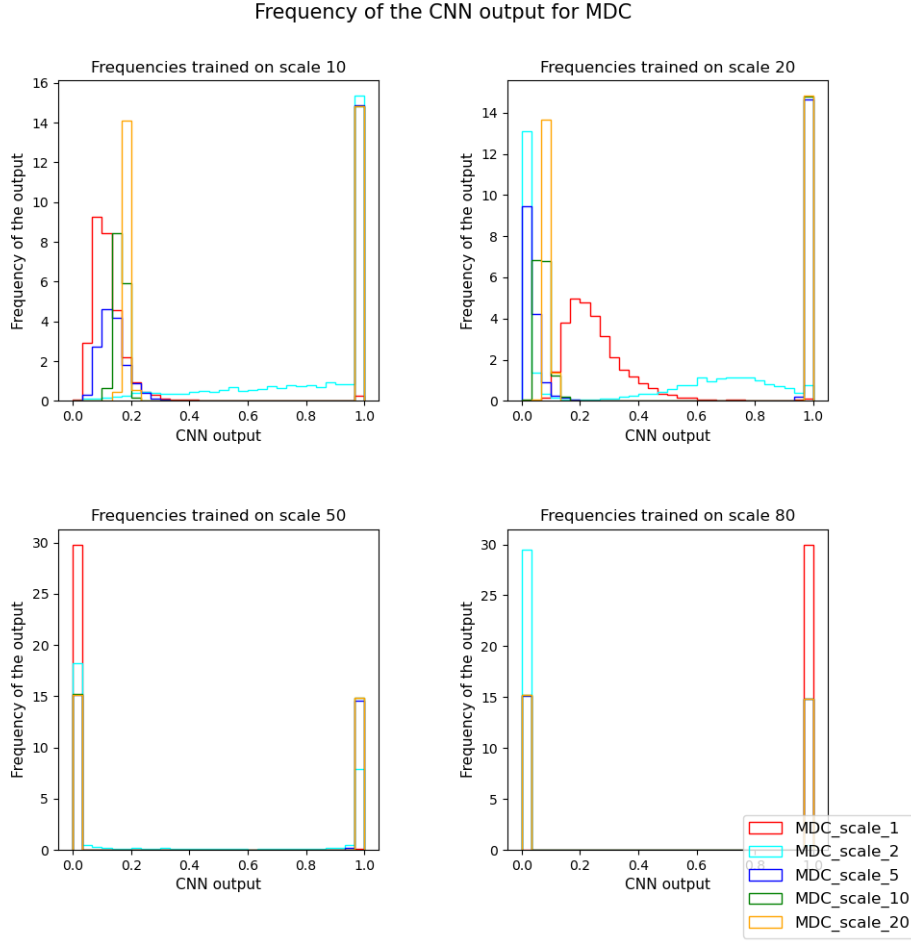


Figure 13: Frequencies of the CNN's output for training on different scales of non-overlapping data, tested on scaled and unscaled MDC data

For trained scale 80 and still tested on MDC scale 2, it basically identifies every spectrogram as noise explaining the very low TPR. This phenomena is well represented with the AUC values (see figure 12) where a sudden drop in values is present, from (0.99, 1.0 and 0.97) for trained scales (10, 20, 50), to 0.05 for trained scale 80. It is also evident on the frequency of the CNN output plot (see figure 13) with a clear concentration of the output around 0. The very low AUC value is indeed implied by the fact that all truly positive data points are classified as negative, which is represented by the CNN's output being solely 0. Thus, the CNN trained on non-overlapping scale 80 data sees every input spectrogram as noise.

For other tested scales (5, 10, 20), we can see that every single CNN trained on scales (10, 50, 80) are doing an excellent job at differentiating mergers and non-mergers for overlapping signals, firstly because of the flat ROC curves having a TPR of 1 for almost all FPR values (see figure 11) but as well as with regard to the AUC values nearly exceeding all AUC values for non-overlapping signals data sets (see figure 12). These tendencies are clearly shown on the frequency of the CNN's output plot (see figure 13), where there is a clear gap, with frequency values of 0, between the two limits of the graph and two peaks with high concentrations around CNN's output values of 0 and 1. This means the CNN has no trouble differentiating between mergers and non-mergers and thus has a high

identification capability. Compared to testing with non-overlapping signals data sets and for training scale 80, the CNN is even performing better for tested MDC scales (5, 10, 20). This is clearly shown in the AUC values table (see figure 12), with way higher values than for non-overlapping signals. Indeed, the frequency of the CNN's output plot (see figure 13) shows a clear separation between two perfectly balanced peaks centered respectively around 0 and 1. This implies that the CNN somehow has learned a perfect threshold to differentiate between both types of data, which must be caused by the nature of the data for scales 50 and 80, where it is very easy (even for a human eye) to tell signal from noise. Therefore, it does a perfect job at separating them.

We can conclude that CNNs trained on high SNR non-overlapping signals have a great performance when tested on high SNR overlapping signals, even when trained on a small scale data set.

5 Comparisons between matched-filtering and CNNs

5.1 The need for other methods than matched-filtering for the future Einstein-Telescope

First of all, MF is the known optimal mathematical method for signal detection in data analysis in the case of non-overlapping signals. But in reality, and like I have explained in section 3.4, ET's data is highly overlapping causing MF with single signal templates to fail (see figure 3). Detection capabilities can thus be higher with CNNs than with single templates MF in the case of overlapping signals data sets. In addition, MF involves a lot of computational resources and is therefore, with the same CPU availability, way slower than an already trained CNN. With such a high frequency of events detected by the future ET (1 per minute instead of 1 per day for previous detectors), other solutions need to be found. One of them is to first train a CNN on a large data bank of simulated signals, which takes time and computational resources, but once trained can perform with lower CPU usage and deal with the large and continuously incoming detector's data sets. Therefore, 1d CNNs could be used preferentially (as found in [2]) for BBH identification, and 2d CNNs for BNS and BHNS.

As a first study and for the reasons explained before, we decided to evaluate the capabilities of both types of CNNs trained solely on BBH, and tested on various data sets including different CBS types.

5.2 Comparisons between results with both methods

We can indeed see that the detection capabilities of CNNs trained on different scales of non-overlapping data exceeds greatly those of single signal MF when using relatively small windows (90s for MF and 16s for CNNs) (see figure 14). The fact there is only about 300 files for evaluation of the performance is due to the fact that 70% of the 1176 files produced were used for training, and the 30% rest for evaluation. For every trained scale (10, 20, 50) and for tested MDC scale 1, a large number of files has a very low fraction of mergers found, which is represented by the red curve with a near constant value around 0. But, we can see that it is still above our computed MF fraction curve, and thus we have the confirmation that low SNR mergers are better detected with CNNs in the case of overlapping signals, even for CNNs trained on high SNRs mergers and small scale data sets. This confirms the choice of direction towards ML and CNNs as solutions to cope with the new data that will be produced at the future ET, when MF and CNNs are using relatively small data windows compared to the ones necessary for BNS and BHNS high detection rate with MF. We can first conclude that BBH, which in principle have high SNR values with MF performed on small data windows, are still better detected with CNNs in the case of overlapping signals. We can also conclude that high SNR mergers of other CBS types are still better detected with CNNs than with MF performed on relatively small window data size.

For other tested MDC scales (2, 5, 10, 20), we find again the conclusions detailed in section 4.6. Indeed, there is a very large fraction of the mergers detected by the CNNs for training scales (10, 20, 80) and tested on scales above 5, represented by the near constant value of 1 curves, which again shows their capabilities of feature extraction transfer between different SNR value mergers. Finally for tested scale 2, there is a great identification capability by the CNN trained on scale 10, but a remarkable drop for other trained scales especially for 50, represented by the curve centered around 60% and

Fractions of the mergers found for MF and ML

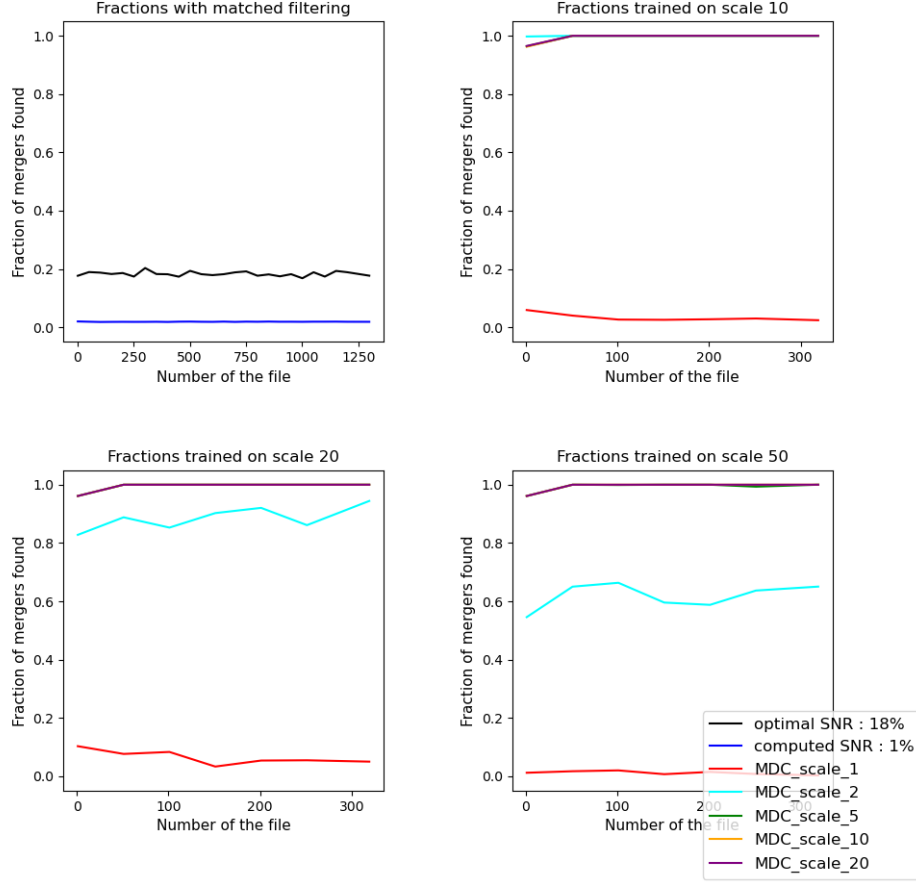


Figure 14: Fractions of the mergers found per file for different tested MDC scales, both with MF and CNNs methods

indicating the CNN’s ability to detect noise, with a low mergers detection rate. To include the full comparison of the detection capabilities between the two methods, we could in the future perform MF on scaled MDC data.

In conclusion, I can say that ML and CNNs are indeed a better solution than MF for overlapping gravitational-waves signals and when considering a restricted short time window (90s for MF and 16s for CNNs). Also, an architecture like Res-Net 50 allows for good feature extraction transferring capabilities between high and mid SNR value mergers. Thus, training on non-overlapping high SNR mergers can be done to obtain great detection capabilities for high and mid SNR mergers, with overlapping or non-overlapping sig-

nals. But to be sure about the extension of these conclusions to BNS and BHNS, we should pursue research development into the use of different window sizes, both for MF and CNNs.

6 Global conclusion and outlooks

The first conclusion is that matched-filtering is a great method when you have large computational resources and non-overlapping signals. Because of the scale of the quantity of data, CPU availability may be lacking for to the future Einstein-Telescope, and the nature of the data is fundamentally overlapping due to the telescope’s detection capabilities and thus the very high

rate of captured mergers events. Therefore, new methods of signal processing and data analysis, like 1d or 2d CNNs, need to be implemented. The second important conclusion is that training on non-overlapping signals can be enough to identify overlapping signals with high associated SNRs values, and the fraction of mergers recovered is indeed higher than with our computed matched-filtering method. Finally, we see that it is possible to transfer feature extraction capabilities between different scales, which means that above a certain SNR value, a CNN trained on statistically higher SNRs mergers still has a good identification rate. But to identify low SNR mergers with a 2d CNN and a better performance, large and various data sets need to be produced in order to let the CNN train longer and better map the features abstract function associated to the input spectrograms. We can thus say that, because MDC data contains a lot of different SNRs value mergers due to the various natures of the CBS (BBH, BNS or BHNS), including an SNR cut in the training implies a statistic limitation of the performance of the 2d CNNs. Therefore, 2d CNNs can be an alternative for quick data analysis, but improvements need to be included in order to extend the detection capabilities to low SNRs mergers and thus unscaled data.

bility to perform parameter inference.

Possible further improvement to the 2d CNN's detection capabilities are firstly to include training on overlapping signals for different SNR thresholds. Another very important perspective is to simply produce more data by creating more templates. Machine Learning methods and their performances are often regulated by the size of the training and development sets, which means the recovery rate of low SNR signals could get much higher. Another aspect that could be experimentally tested is the empirical modification of the CNN's structure and layer parameters, like the number of layers or the kernel and filter sizes. This could, for example, allow for the inclusion of frequencies above 1000 Hz and still capture much of the important features associated to a merger spectrogram. As the study ([2]) shows, 1d CNNs can be better at detecting BBH and 2d CNNs for BNS and BHNS due to the differences in their respective waveforms. We could therefore further develop each of the types of CNN and then create a combinatory version of them. In order to fully compare the detection capabilities between matched-filtering and 2d CNNs, we could also perform MF on scaled data. Finally, one of the most important further development, necessary for the Einstein Telescope, is to add the capa-

References

- [1] Andrew Ng et al. *Deep learning Specialization : Coursera MOOC*. <https://www.coursera.org/specializations/deep-learning>. 2024.
- [2] Cornell University : see for authors Arxiv. *Comparative study of 1D and 2D convolutional neural network models with attribution analysis for gravitational wave detection from compact binary coalescences*. <https://arxiv.org/abs/2312.04855>. 2024.
- [3] Physical Review : see for authors. *Can we measure individual black-hole spins from gravitational-wave observations?* <https://journals.aps.org/prd/abstract/10.1103/PhysRevD.93.084042>. 2016.
- [4] Physical Review : see for authors. *Frequency-domain gravitational waves from nonprecessing black-hole binaries. II. A phenomenological model for the advanced detector era*. <https://journals.aps.org/prd/abstract/10.1103/PhysRevD.93.044007>. 2016.
- [5] GWOSC. *GWOSC Course*. <https://gwosc.org>. 2024.
- [6] Gwpy. *Gwpy : github*. <https://gwpy.github.io/>. 2024.
- [7] Gwpy. *Gwpy : pypi*. <https://pypi.org/project/gwpy/>. 2024.
- [8] Gwpy. *Gwpy : spectrogram*. <https://gwpy.github.io/docs/stable/examples/spectrogram/spectrogram2/>. 2024.
- [9] Michele Maggiore. *Gravitational waves : Volume 1, Theory and experiments*. Oxford University Press, 2007.
- [10] Michele Maggiore. *Gravitational waves : Volume 2, Astrophysics and Cosmology*. Oxford University Press, 2018.
- [11] Pycbc. *Pycbc website : detector*. <https://pycbc.org/pycbc/latest/html/detector.html>. 2024.
- [12] Pycbc. *Pycbc website : waveforms*. https://pycbc.org/pycbc/latest/html/pycbc_waveform.html. 2024.
- [13] Roboflow. *What is ResNet-50*. <https://blog.roboflow.com/what-is-resnet-50/>. 2024.
- [14] Noe Scharer. *My code*. https://drive.google.com/drive/folders/1tR8YtqQy8jgTaVIy8E79AW0clJUrAgCf?usp=share_link. 2024.
- [15] Einstein Telescope. *ET-CE MDC*. <https://docs.google.com/document/d/1tSQtca9VgUF7DL5N5Xtp1lDiH9f1ZaVEWkkyD8wL-pI/edit>. 2024.

7 Annex

Last login: Thu Aug 1 10:42:54 on ttys001

The default interactive shell is now zsh.

To update your account to use zsh, please run `chsh -s /bin/zsh`.

For more details, please visit <https://support.apple.com/kb/HT208050>.

(base) macbook-pro-de-corinne-charbonnel:~ noeschaerer\$ conda activate igwn-py39-lw

(igwn-py39-lw) macbook-pro-de-corinne-charbonnel:~ noeschaerer\$ python odw_2023_main/plot_final.py

Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 128, 256, 1)	0	-
zero_padding2d (ZeroPadding2D)	(None, 134, 262, 1)	0	input_layer[0][0]
conv2d (Conv2D)	(None, 60, 119, 64)	25,664	zero_padding2d[0][0]
batch_normalization (BatchNormalization)	(None, 60, 119, 64)	256	conv2d[0][0]
activation (Activation)	(None, 60, 119, 64)	0	batch_normalization[0][0]
max_pooling2d (MaxPooling2D)	(None, 29, 59, 64)	0	activation[0][0]
dropout (Dropout)	(None, 29, 59, 64)	0	max_pooling2d[0][0]
conv2d_1 (Conv2D)	(None, 29, 59, 64)	4,160	dropout[0][0]
batch_normalization_1 (BatchNormalization)	(None, 29, 59, 64)	256	conv2d_1[0][0]
activation_1 (Activation)	(None, 29, 59, 64)	0	batch_normalization_1[0][...
dropout_1 (Dropout)	(None, 29, 59, 64)	0	activation_1[0][0]
conv2d_2 (Conv2D)	(None, 29, 59, 64)	36,928	dropout_1[0][0]
batch_normalization_2 (BatchNormalization)	(None, 29, 59, 64)	256	conv2d_2[0][0]
activation_2 (Activation)	(None, 29, 59, 64)	0	batch_normalization_2[0][...
dropout_2 (Dropout)	(None, 29, 59, 64)	0	activation_2[0][0]
conv2d_3 (Conv2D)	(None, 29, 59, 256)	16,640	dropout_2[0][0]
batch_normalization_3 (BatchNormalization)	(None, 29, 59, 256)	1,024	conv2d_3[0][0]
dropout_3 (Dropout)	(None, 29, 59, 256)	0	batch_normalization_3[0][...
conv2d_4 (Conv2D)	(None, 29, 59, 256)	16,640	dropout_3[0][0]
dropout_4 (Dropout)	(None, 29, 59, 256)	0	dropout_3[0][0]
batch_normalization_4 (BatchNormalization)	(None, 29, 59, 256)	1,024	conv2d_4[0][0]
add (Add)	(None, 29, 59, 256)	0	dropout_4[0][0], batch_normalization_4[0][...
activation_3 (Activation)	(None, 29, 59, 256)	0	add[0][0]
conv2d_5 (Conv2D)	(None, 29, 59, 64)	16,448	activation_3[0][0]
batch_normalization_5 (BatchNormalization)	(None, 29, 59, 64)	256	conv2d_5[0][0]
activation_4 (Activation)	(None, 29, 59, 64)	0	batch_normalization_5[0][...
dropout_5 (Dropout)	(None, 29, 59, 64)	0	activation_4[0][0]
conv2d_6 (Conv2D)	(None, 29, 59, 64)	36,928	dropout_5[0][0]
batch_normalization_6 (BatchNormalization)	(None, 29, 59, 64)	256	conv2d_6[0][0]
activation_5 (Activation)	(None, 29, 59, 64)	0	batch_normalization_6[0][...
dropout_6 (Dropout)	(None, 29, 59, 64)	0	activation_5[0][0]
conv2d_7 (Conv2D)	(None, 29, 59, 256)	16,640	dropout_6[0][0]
batch_normalization_7 (BatchNormalization)	(None, 29, 59, 256)	1,024	conv2d_7[0][0]
dropout_7 (Dropout)	(None, 29, 59, 256)	0	batch_normalization_7[0][...
add_1 (Add)	(None, 29, 59, 256)	0	activation_3[0][0], dropout_7[0][0]
activation_6 (Activation)	(None, 29, 59, 256)	0	add_1[0][0]
conv2d_8 (Conv2D)	(None, 29, 59, 64)	16,448	activation_6[0][0]
batch_normalization_8 (BatchNormalization)	(None, 29, 59, 64)	256	conv2d_8[0][0]
activation_7 (Activation)	(None, 29, 59, 64)	0	batch_normalization_8[0][...

dropout_8 (Dropout)	(None, 29, 59, 64)	0	activation_7[0][0]
conv2d_9 (Conv2D)	(None, 29, 59, 64)	36,928	dropout_8[0][0]
batch_normalization_9 (BatchNormalization)	(None, 29, 59, 64)	256	conv2d_9[0][0]
activation_8 (Activation)	(None, 29, 59, 64)	0	batch_normalization_9[0][...]
dropout_9 (Dropout)	(None, 29, 59, 64)	0	activation_8[0][0]
conv2d_10 (Conv2D)	(None, 29, 59, 256)	16,640	dropout_9[0][0]
batch_normalization_10 (BatchNormalization)	(None, 29, 59, 256)	1,024	conv2d_10[0][0]
dropout_10 (Dropout)	(None, 29, 59, 256)	0	batch_normalization_10[0]...
add_2 (Add)	(None, 29, 59, 256)	0	activation_6[0][0], dropout_10[0][0]
activation_9 (Activation)	(None, 29, 59, 256)	0	add_2[0][0]
dropout_11 (Dropout)	(None, 29, 59, 256)	0	activation_9[0][0]
conv2d_11 (Conv2D)	(None, 15, 30, 128)	32,896	dropout_11[0][0]
batch_normalization_11 (BatchNormalization)	(None, 15, 30, 128)	512	conv2d_11[0][0]
activation_10 (Activation)	(None, 15, 30, 128)	0	batch_normalization_11[0]...
dropout_12 (Dropout)	(None, 15, 30, 128)	0	activation_10[0][0]
conv2d_12 (Conv2D)	(None, 15, 30, 128)	147,584	dropout_12[0][0]
batch_normalization_12 (BatchNormalization)	(None, 15, 30, 128)	512	conv2d_12[0][0]
activation_11 (Activation)	(None, 15, 30, 128)	0	batch_normalization_12[0]...
dropout_13 (Dropout)	(None, 15, 30, 128)	0	activation_11[0][0]
conv2d_13 (Conv2D)	(None, 15, 30, 512)	66,048	dropout_13[0][0]
batch_normalization_13 (BatchNormalization)	(None, 15, 30, 512)	2,048	conv2d_13[0][0]
dropout_14 (Dropout)	(None, 15, 30, 512)	0	batch_normalization_13[0]...
conv2d_14 (Conv2D)	(None, 15, 30, 512)	131,584	dropout_11[0][0]
dropout_15 (Dropout)	(None, 15, 30, 512)	0	dropout_14[0][0]
batch_normalization_14 (BatchNormalization)	(None, 15, 30, 512)	2,048	conv2d_14[0][0]
add_3 (Add)	(None, 15, 30, 512)	0	dropout_15[0][0], batch_normalization_14[0]...
activation_12 (Activation)	(None, 15, 30, 512)	0	add_3[0][0]
dropout_16 (Dropout)	(None, 15, 30, 512)	0	activation_12[0][0]
conv2d_15 (Conv2D)	(None, 15, 30, 128)	65,664	dropout_16[0][0]
batch_normalization_15 (BatchNormalization)	(None, 15, 30, 128)	512	conv2d_15[0][0]
activation_13 (Activation)	(None, 15, 30, 128)	0	batch_normalization_15[0]...
dropout_17 (Dropout)	(None, 15, 30, 128)	0	activation_13[0][0]
conv2d_16 (Conv2D)	(None, 15, 30, 128)	147,584	dropout_17[0][0]
batch_normalization_16 (BatchNormalization)	(None, 15, 30, 128)	512	conv2d_16[0][0]
activation_14 (Activation)	(None, 15, 30, 128)	0	batch_normalization_16[0]...
dropout_18 (Dropout)	(None, 15, 30, 128)	0	activation_14[0][0]
conv2d_17 (Conv2D)	(None, 15, 30, 512)	66,048	dropout_18[0][0]
batch_normalization_17 (BatchNormalization)	(None, 15, 30, 512)	2,048	conv2d_17[0][0]
dropout_19 (Dropout)	(None, 15, 30, 512)	0	batch_normalization_17[0]...
add_4 (Add)	(None, 15, 30, 512)	0	dropout_16[0][0], dropout_19[0][0]
activation_15 (Activation)	(None, 15, 30, 512)	0	add_4[0][0]
conv2d_18 (Conv2D)	(None, 15, 30, 128)	65,664	activation_15[0][0]
batch_normalization_18 (BatchNormalization)	(None, 15, 30, 128)	512	conv2d_18[0][0]

activation_16 (Activation)	(None, 15, 30, 128)	0	batch_normalization_18[0]...
dropout_20 (Dropout)	(None, 15, 30, 128)	0	activation_16[0][0]
conv2d_19 (Conv2D)	(None, 15, 30, 128)	147,584	dropout_20[0][0]
batch_normalization_19 (BatchNormalization)	(None, 15, 30, 128)	512	conv2d_19[0][0]
activation_17 (Activation)	(None, 15, 30, 128)	0	batch_normalization_19[0]...
dropout_21 (Dropout)	(None, 15, 30, 128)	0	activation_17[0][0]
conv2d_20 (Conv2D)	(None, 15, 30, 512)	66,048	dropout_21[0][0]
batch_normalization_20 (BatchNormalization)	(None, 15, 30, 512)	2,048	conv2d_20[0][0]
dropout_22 (Dropout)	(None, 15, 30, 512)	0	batch_normalization_20[0]...
add_5 (Add)	(None, 15, 30, 512)	0	activation_15[0][0], dropout_22[0][0]
activation_18 (Activation)	(None, 15, 30, 512)	0	add_5[0][0]
conv2d_21 (Conv2D)	(None, 15, 30, 128)	65,664	activation_18[0][0]
batch_normalization_21 (BatchNormalization)	(None, 15, 30, 128)	512	conv2d_21[0][0]
activation_19 (Activation)	(None, 15, 30, 128)	0	batch_normalization_21[0]...
dropout_23 (Dropout)	(None, 15, 30, 128)	0	activation_19[0][0]
conv2d_22 (Conv2D)	(None, 15, 30, 128)	147,584	dropout_23[0][0]
batch_normalization_22 (BatchNormalization)	(None, 15, 30, 128)	512	conv2d_22[0][0]
activation_20 (Activation)	(None, 15, 30, 128)	0	batch_normalization_22[0]...
dropout_24 (Dropout)	(None, 15, 30, 128)	0	activation_20[0][0]
conv2d_23 (Conv2D)	(None, 15, 30, 512)	66,048	dropout_24[0][0]
batch_normalization_23 (BatchNormalization)	(None, 15, 30, 512)	2,048	conv2d_23[0][0]
dropout_25 (Dropout)	(None, 15, 30, 512)	0	batch_normalization_23[0]...
add_6 (Add)	(None, 15, 30, 512)	0	activation_18[0][0], dropout_25[0][0]
activation_21 (Activation)	(None, 15, 30, 512)	0	add_6[0][0]
dropout_26 (Dropout)	(None, 15, 30, 512)	0	activation_21[0][0]
conv2d_24 (Conv2D)	(None, 8, 15, 256)	131,328	dropout_26[0][0]
batch_normalization_24 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_24[0][0]
activation_22 (Activation)	(None, 8, 15, 256)	0	batch_normalization_24[0]...
dropout_27 (Dropout)	(None, 8, 15, 256)	0	activation_22[0][0]
conv2d_25 (Conv2D)	(None, 8, 15, 256)	590,080	dropout_27[0][0]
batch_normalization_25 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_25[0][0]
activation_23 (Activation)	(None, 8, 15, 256)	0	batch_normalization_25[0]...
dropout_28 (Dropout)	(None, 8, 15, 256)	0	activation_23[0][0]
conv2d_26 (Conv2D)	(None, 8, 15, 1024)	263,168	dropout_28[0][0]
batch_normalization_26 (BatchNormalization)	(None, 8, 15, 1024)	4,096	conv2d_26[0][0]
dropout_29 (Dropout)	(None, 8, 15, 1024)	0	batch_normalization_26[0]...
conv2d_27 (Conv2D)	(None, 8, 15, 1024)	525,312	dropout_26[0][0]
dropout_30 (Dropout)	(None, 8, 15, 1024)	0	dropout_29[0][0]
batch_normalization_27 (BatchNormalization)	(None, 8, 15, 1024)	4,096	conv2d_27[0][0]
add_7 (Add)	(None, 8, 15, 1024)	0	dropout_30[0][0], batch_normalization_27[0]...
activation_24 (Activation)	(None, 8, 15, 1024)	0	add_7[0][0]
dropout_31 (Dropout)	(None, 8, 15, 1024)	0	activation_24[0][0]
conv2d_28 (Conv2D)	(None, 8, 15, 256)	262,400	dropout_31[0][0]
batch_normalization_28 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_28[0][0]

activation_25 (Activation)	(None, 8, 15, 256)	0	batch_normalization_28[0]...
dropout_32 (Dropout)	(None, 8, 15, 256)	0	activation_25[0][0]
conv2d_29 (Conv2D)	(None, 8, 15, 256)	590,080	dropout_32[0][0]
batch_normalization_29 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_29[0][0]
activation_26 (Activation)	(None, 8, 15, 256)	0	batch_normalization_29[0]...
dropout_33 (Dropout)	(None, 8, 15, 256)	0	activation_26[0][0]
conv2d_30 (Conv2D)	(None, 8, 15, 1024)	263,168	dropout_33[0][0]
batch_normalization_30 (BatchNormalization)	(None, 8, 15, 1024)	4,096	conv2d_30[0][0]
dropout_34 (Dropout)	(None, 8, 15, 1024)	0	batch_normalization_30[0]...
add_8 (Add)	(None, 8, 15, 1024)	0	dropout_31[0][0], dropout_34[0][0]
activation_27 (Activation)	(None, 8, 15, 1024)	0	add_8[0][0]
conv2d_31 (Conv2D)	(None, 8, 15, 256)	262,400	activation_27[0][0]
batch_normalization_31 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_31[0][0]
activation_28 (Activation)	(None, 8, 15, 256)	0	batch_normalization_31[0]...
dropout_35 (Dropout)	(None, 8, 15, 256)	0	activation_28[0][0]
conv2d_32 (Conv2D)	(None, 8, 15, 256)	590,080	dropout_35[0][0]
batch_normalization_32 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_32[0][0]
activation_29 (Activation)	(None, 8, 15, 256)	0	batch_normalization_32[0]...
dropout_36 (Dropout)	(None, 8, 15, 256)	0	activation_29[0][0]
conv2d_33 (Conv2D)	(None, 8, 15, 1024)	263,168	dropout_36[0][0]
batch_normalization_33 (BatchNormalization)	(None, 8, 15, 1024)	4,096	conv2d_33[0][0]
dropout_37 (Dropout)	(None, 8, 15, 1024)	0	batch_normalization_33[0]...
add_9 (Add)	(None, 8, 15, 1024)	0	activation_27[0][0], dropout_37[0][0]
activation_30 (Activation)	(None, 8, 15, 1024)	0	add_9[0][0]
conv2d_34 (Conv2D)	(None, 8, 15, 256)	262,400	activation_30[0][0]
batch_normalization_34 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_34[0][0]
activation_31 (Activation)	(None, 8, 15, 256)	0	batch_normalization_34[0]...
dropout_38 (Dropout)	(None, 8, 15, 256)	0	activation_31[0][0]
conv2d_35 (Conv2D)	(None, 8, 15, 256)	590,080	dropout_38[0][0]
batch_normalization_35 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_35[0][0]
activation_32 (Activation)	(None, 8, 15, 256)	0	batch_normalization_35[0]...
dropout_39 (Dropout)	(None, 8, 15, 256)	0	activation_32[0][0]
conv2d_36 (Conv2D)	(None, 8, 15, 1024)	263,168	dropout_39[0][0]
batch_normalization_36 (BatchNormalization)	(None, 8, 15, 1024)	4,096	conv2d_36[0][0]
dropout_40 (Dropout)	(None, 8, 15, 1024)	0	batch_normalization_36[0]...
add_10 (Add)	(None, 8, 15, 1024)	0	activation_30[0][0], dropout_40[0][0]
activation_33 (Activation)	(None, 8, 15, 1024)	0	add_10[0][0]
conv2d_37 (Conv2D)	(None, 8, 15, 256)	262,400	activation_33[0][0]
batch_normalization_37 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_37[0][0]
activation_34 (Activation)	(None, 8, 15, 256)	0	batch_normalization_37[0]...
dropout_41 (Dropout)	(None, 8, 15, 256)	0	activation_34[0][0]
conv2d_38 (Conv2D)	(None, 8, 15, 256)	590,080	dropout_41[0][0]
batch_normalization_38 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_38[0][0]

activation_35 (Activation)	(None, 8, 15, 256)	0	batch_normalization_38[0]...
dropout_42 (Dropout)	(None, 8, 15, 256)	0	activation_35[0][0]
conv2d_39 (Conv2D)	(None, 8, 15, 1024)	263,168	dropout_42[0][0]
batch_normalization_39 (BatchNormalization)	(None, 8, 15, 1024)	4,096	conv2d_39[0][0]
dropout_43 (Dropout)	(None, 8, 15, 1024)	0	batch_normalization_39[0]...
add_11 (Add)	(None, 8, 15, 1024)	0	activation_33[0][0], dropout_43[0][0]
activation_36 (Activation)	(None, 8, 15, 1024)	0	add_11[0][0]
conv2d_40 (Conv2D)	(None, 8, 15, 256)	262,400	activation_36[0][0]
batch_normalization_40 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_40[0][0]
activation_37 (Activation)	(None, 8, 15, 256)	0	batch_normalization_40[0]...
dropout_44 (Dropout)	(None, 8, 15, 256)	0	activation_37[0][0]
conv2d_41 (Conv2D)	(None, 8, 15, 256)	590,080	dropout_44[0][0]
batch_normalization_41 (BatchNormalization)	(None, 8, 15, 256)	1,024	conv2d_41[0][0]
activation_38 (Activation)	(None, 8, 15, 256)	0	batch_normalization_41[0]...
dropout_45 (Dropout)	(None, 8, 15, 256)	0	activation_38[0][0]
conv2d_42 (Conv2D)	(None, 8, 15, 1024)	263,168	dropout_45[0][0]
batch_normalization_42 (BatchNormalization)	(None, 8, 15, 1024)	4,096	conv2d_42[0][0]
dropout_46 (Dropout)	(None, 8, 15, 1024)	0	batch_normalization_42[0]...
add_12 (Add)	(None, 8, 15, 1024)	0	activation_36[0][0], dropout_46[0][0]
activation_39 (Activation)	(None, 8, 15, 1024)	0	add_12[0][0]
dropout_47 (Dropout)	(None, 8, 15, 1024)	0	activation_39[0][0]
conv2d_43 (Conv2D)	(None, 4, 8, 512)	524,800	dropout_47[0][0]
batch_normalization_43 (BatchNormalization)	(None, 4, 8, 512)	2,048	conv2d_43[0][0]
activation_40 (Activation)	(None, 4, 8, 512)	0	batch_normalization_43[0]...
dropout_48 (Dropout)	(None, 4, 8, 512)	0	activation_40[0][0]
conv2d_44 (Conv2D)	(None, 4, 8, 512)	2,359,808	dropout_48[0][0]
batch_normalization_44 (BatchNormalization)	(None, 4, 8, 512)	2,048	conv2d_44[0][0]
activation_41 (Activation)	(None, 4, 8, 512)	0	batch_normalization_44[0]...
dropout_49 (Dropout)	(None, 4, 8, 512)	0	activation_41[0][0]
conv2d_45 (Conv2D)	(None, 4, 8, 2048)	1,050,624	dropout_49[0][0]
batch_normalization_45 (BatchNormalization)	(None, 4, 8, 2048)	8,192	conv2d_45[0][0]
dropout_50 (Dropout)	(None, 4, 8, 2048)	0	batch_normalization_45[0]...
conv2d_46 (Conv2D)	(None, 4, 8, 2048)	2,099,200	dropout_47[0][0]
dropout_51 (Dropout)	(None, 4, 8, 2048)	0	dropout_50[0][0]
batch_normalization_46 (BatchNormalization)	(None, 4, 8, 2048)	8,192	conv2d_46[0][0]
add_13 (Add)	(None, 4, 8, 2048)	0	dropout_51[0][0], batch_normalization_46[0]...
activation_42 (Activation)	(None, 4, 8, 2048)	0	add_13[0][0]
dropout_52 (Dropout)	(None, 4, 8, 2048)	0	activation_42[0][0]
conv2d_47 (Conv2D)	(None, 4, 8, 512)	1,049,088	dropout_52[0][0]
batch_normalization_47 (BatchNormalization)	(None, 4, 8, 512)	2,048	conv2d_47[0][0]
activation_43 (Activation)	(None, 4, 8, 512)	0	batch_normalization_47[0]...
dropout_53 (Dropout)	(None, 4, 8, 512)	0	activation_43[0][0]
conv2d_48 (Conv2D)	(None, 4, 8, 512)	2,359,808	dropout_53[0][0]
batch_normalization_48 (BatchNormalization)	(None, 4, 8, 512)	2,048	conv2d_48[0][0]

activation_44 (Activation)	(None, 4, 8, 512)	0	batch_normalization_48[0]...
dropout_54 (Dropout)	(None, 4, 8, 512)	0	activation_44[0][0]
conv2d_49 (Conv2D)	(None, 4, 8, 2048)	1,050,624	dropout_54[0][0]
batch_normalization_49 (BatchNormalization)	(None, 4, 8, 2048)	8,192	conv2d_49[0][0]
dropout_55 (Dropout)	(None, 4, 8, 2048)	0	batch_normalization_49[0]...
add_14 (Add)	(None, 4, 8, 2048)	0	dropout_52[0][0], dropout_55[0][0]
activation_45 (Activation)	(None, 4, 8, 2048)	0	add_14[0][0]
conv2d_50 (Conv2D)	(None, 4, 8, 512)	1,049,088	activation_45[0][0]
batch_normalization_50 (BatchNormalization)	(None, 4, 8, 512)	2,048	conv2d_50[0][0]
activation_46 (Activation)	(None, 4, 8, 512)	0	batch_normalization_50[0]...
dropout_56 (Dropout)	(None, 4, 8, 512)	0	activation_46[0][0]
conv2d_51 (Conv2D)	(None, 4, 8, 512)	2,359,808	dropout_56[0][0]
batch_normalization_51 (BatchNormalization)	(None, 4, 8, 512)	2,048	conv2d_51[0][0]
activation_47 (Activation)	(None, 4, 8, 512)	0	batch_normalization_51[0]...
dropout_57 (Dropout)	(None, 4, 8, 512)	0	activation_47[0][0]
conv2d_52 (Conv2D)	(None, 4, 8, 2048)	1,050,624	dropout_57[0][0]
batch_normalization_52 (BatchNormalization)	(None, 4, 8, 2048)	8,192	conv2d_52[0][0]
dropout_58 (Dropout)	(None, 4, 8, 2048)	0	batch_normalization_52[0]...
add_15 (Add)	(None, 4, 8, 2048)	0	activation_45[0][0], dropout_58[0][0]
activation_48 (Activation)	(None, 4, 8, 2048)	0	add_15[0][0]
dropout_59 (Dropout)	(None, 4, 8, 2048)	0	activation_48[0][0]
average_pooling2d (AveragePooling2D)	(None, 2, 4, 2048)	0	dropout_59[0][0]
flatten (Flatten)	(None, 16384)	0	average_pooling2d[0][0]
dense (Dense)	(None, 1)	16,385	flatten[0][0]
dropout_60 (Dropout)	(None, 1)	0	dense[0][0]

Total params: 23,620,289 (90.10 MB)

Trainable params: 23,567,169 (89.90 MB)

Non-trainable params: 53,120 (207.50 KB)

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

(igwn-py39-lw) macbook-pro-de-corinne-charbonnel:~ noeschaerer\$